

Introduction

Containerized applications require a mechanism to deploy, schedule, network, scale, and maintain workloads consistently across infrastructure. **Kubernetes, Docker Swarm, and AWS Fargate address these requirements through substantially different architectural models.**

Kubernetes is a full-featured, extensible container orchestration platform. Docker Swarm provides orchestration capabilities directly within Docker Engine, emphasizing a comparatively straightforward operational model. AWS Fargate takes a different approach: it provides managed, serverless compute for containers running through Amazon ECS or Amazon EKS, abstracting away the underlying compute instances.

This report compares the three technologies across their **architecture, scalability, operational complexity, and cost structure**, using the supplied technical and industry facts as the basis. The comparison focuses on documented characteristics rather than subjective product preferences.

Tech Overviews

Kubernetes

Kubernetes uses a **control-plane/worker-node architecture**. The control plane contains the API server, `etcd`, scheduler, controller manager, and optionally the cloud-controller manager. Worker nodes run components such as the kubelet, networking components, and a container runtime.

Its architecture is designed around **declarative desired state and continuous reconciliation**. Users describe the desired state of workloads, while Kubernetes controllers work to make the actual cluster state conform to that specification.

A major characteristic of Kubernetes is its extensibility. Networking, storage, authentication, observability, ingress, security, and cloud integrations can be provided through a broad ecosystem of components and extensions.

For Kubernetes v1.37, the documented large-cluster design targets include **5,000 nodes, 150,000 Pods, and 300,000 containers per cluster**, with a recommended maximum of 110 Pods per node. These are project design targets rather than universal physical limits.

The trade-off for this breadth is operational complexity. A self-managed production cluster requires control-plane infrastructure, worker nodes, networking, storage, security configuration, monitoring, backups, and ongoing cluster administration. Managed Kubernetes services reduce some of the control-plane burden but do not remove workload-level configuration.

Kubernetes itself carries **no upstream software licensing fee**. Costs instead arise from infrastructure and operational resources, or from fees charged by a cloud provider for a managed Kubernetes service.

Kubernetes also has substantial industry adoption. The supplied CNCF 2025 Annual Cloud Native Survey reported that **82% of container users surveyed were running Kubernetes in production**, while 98% reported adopting cloud-native techniques. These figures describe the survey population rather than the entire IT industry.

Docker Swarm

Docker Swarm, or Swarm mode, is integrated directly into **Docker Engine**. Rather than introducing a separate orchestration platform, it extends Docker Engine with clustering and service-management capabilities.

Its architecture consists primarily of **manager and worker nodes**. Managers maintain cluster state, schedule services, expose the Swarm API, and use **Raft consensus** for state management. Worker nodes execute the tasks assigned to them.

The fundamental deployment object is a **service**, which specifies the desired state of a workload, including its image, replica count, resources, networking, placement constraints, and update behavior.

Swarm includes several orchestration capabilities directly, including overlay networking, service discovery, internal load balancing, mutual TLS, rolling updates, replication, and reconciliation.

Docker does not publish a Kubernetes-style hard cluster target such as 5,000 nodes or 150,000 Pods. Instead, its documentation provides architectural guidance, notably recommending **no more than seven manager nodes**. Increasing manager count improves Raft fault tolerance but does not inherently increase workload capacity or performance.

For example, three managers can tolerate one simultaneous manager failure, while five can tolerate two and seven can tolerate three. These characteristics derive from the quorum requirements of Raft.

Setup is comparatively centered on the existing Docker Engine workflow: initialize a manager and join other Docker Engine instances as workers or managers. The basic deployment model therefore involves fewer distinct infrastructure components than a self-managed Kubernetes installation.

Swarm itself is part of the open-source Docker Engine technology. Infrastructure costs consequently come primarily from the machines, networking, storage, registry, monitoring, and operational resources used to host the Swarm.

AWS Fargate

AWS Fargate differs fundamentally from both Kubernetes and Swarm because it is **not itself a standalone container orchestrator**. It is a serverless container compute engine used with **Amazon ECS and Amazon EKS**.

With Fargate, AWS manages the underlying compute infrastructure. The customer specifies the container workload and its resource requirements without having to provision and maintain the underlying EC2 instances for those workloads.

The primary resource model is task/Pod oriented. Fargate supports defined CPU and memory combinations, ranging from relatively small workloads such as **0.25 vCPU with 0.5–2 GB memory** through configurations reaching **32 vCPUs and up to 244 GB memory**, subject to platform and operating-system constraints.

Fargate scaling is constrained by AWS service quotas, including concurrent Fargate vCPU resources and task-launch rates. Some quotas can be adjusted, meaning practical scaling depends not only on the workload but also on the applicable AWS account and regional limits.

Operationally, Fargate removes server provisioning and much of the host-management responsibility. However, users still configure the surrounding container platform, including ECS/EKS resources, IAM, VPC networking, security groups, load balancing, logging, service discovery, and autoscaling.

Fargate uses **usage-based pricing** rather than requiring customers to pay for provisioned container hosts. Billing is based on resources consumed by running workloads, including vCPU, memory, operating system, CPU architecture, and applicable ephemeral storage.

Detailed Comparison

Architectural model

The most fundamental difference is **where orchestration and infrastructure responsibility reside**.

Kubernetes provides a complete orchestration control plane. It determines where workloads run, maintains desired state, handles service abstractions, and coordinates the cluster. The underlying infrastructure can be physical servers, virtual machines, or cloud resources.

Docker Swarm provides a similar orchestration role but embeds the functionality inside Docker Engine. Its service-oriented architecture and manager/worker model provide cluster scheduling and reconciliation without requiring the broader collection of components associated with a typical Kubernetes deployment.

Fargate moves the infrastructure boundary considerably further toward the cloud provider. Fargate supplies compute capacity, while ECS or EKS provides the orchestration layer. Consequently, comparing "Fargate versus Kubernetes" directly requires distinguishing Fargate from **EKS**, because Fargate is a compute mechanism that can actually be used to run EKS Pods.

In simplified architectural terms:

- **Kubernetes:** customer-managed or provider-managed orchestration + customer-selected compute.
- **Docker Swarm:** Docker Engine-based orchestration + customer-managed compute.
- **Fargate:** AWS-managed container compute + ECS/EKS orchestration.

Scalability

Kubernetes has the clearest published large-cluster specifications among the three. Its v1.37 documentation identifies targets of **5,000 nodes, 150,000 Pods, and 300,000 containers per cluster**. Kubernetes can also horizontally scale API-server instances and add or remove worker nodes.

Docker Swarm does not provide an equivalent published maximum workload figure in the supplied documentation. Its scaling architecture is instead influenced by the manager quorum and Raft state-management model. Docker recommends limiting managers to seven, because additional managers increase consensus overhead rather than simply increasing application capacity.

Fargate's scaling model is different again. Customers do not directly scale a fleet of container hosts. Instead, workloads consume AWS-managed compute resources, with scaling constrained by **Fargate/ECS service quotas**, resource combinations, task-launch rates, and regional/account limits.

Therefore, the three systems express scalability through different mechanisms:

Kubernetes → cluster and workload capacity

Swarm → node capacity and manager consensus

Fargate → managed resource consumption and AWS quotas

Importantly, none of these figures should be interpreted as a universal performance ceiling. Actual scalability depends on workload characteristics, networking, storage, API activity, scheduling behavior, regional infrastructure, and configuration.

Setup and operational complexity

A self-managed Kubernetes environment has the largest number of infrastructure components to configure and maintain. A production-oriented deployment may require control-plane high availability, etcd management, networking, storage, identity, security, observability, upgrades, backups, and workload configuration.

This complexity is partly a consequence of Kubernetes' broad functionality and extensibility.

Docker Swarm reduces the initial infrastructure model considerably. Since Swarm mode is built into Docker Engine, administrators primarily work with Docker Engine nodes and Docker's service-management commands. A basic multi-node deployment can therefore be established without installing a separate orchestration control plane.

Fargate removes a different category of complexity: **host management**. Customers do not need to provision or patch the underlying compute instances used for the Fargate workload. Nevertheless, Fargate does not eliminate platform configuration. ECS/EKS, networking, IAM, security, logging, load balancing, and autoscaling remain relevant.

Thus, "complexity" has different sources:

- Kubernetes: **cluster/platform administration**
- Swarm: **cluster administration with an integrated Docker model**
- Fargate: **cloud/application configuration rather than host provisioning**

Resource model

Kubernetes schedules **Pods** onto worker nodes. The cluster administrator ultimately manages the compute environment or relies on a managed Kubernetes provider to do so.

Swarm schedules **service tasks** onto Docker Engine nodes. Capacity therefore depends directly on the available worker infrastructure.

Fargate instead allocates resources at the **task/Pod level**. The user specifies CPU and memory combinations, and AWS handles the underlying host infrastructure.

This creates an important distinction between infrastructure capacity and workload capacity. Kubernetes and Swarm expose the underlying node layer more directly, while Fargate largely abstracts it.

Cost structure

None of the three should be evaluated solely by software licensing cost.

Kubernetes is open source, so the upstream software has no license fee. A self-managed installation incurs infrastructure and engineering costs. Managed Kubernetes adds provider-specific control-plane and infrastructure charges.

Docker Swarm is incorporated into Docker Engine, with infrastructure costs primarily determined by the machines and supporting services used to operate the cluster. Docker's commercial offerings and Docker Desktop licensing requirements are separate considerations from the open-source Swarm technology.

Fargate uses a consumption-based model. Customers pay for the resources associated with their running tasks/Pods rather than managing a fleet of underlying container hosts. AWS

charges according to dimensions such as vCPU and memory, with additional applicable storage and data-transfer charges.

The supplied AWS example illustrates this model using US East (N. Virginia) Linux/x86 pricing of **\$0.000011244 per vCPU-second** and **\$0.000001235 per GB-memory-second**, plus applicable ephemeral-storage charges. These rates are regional and configuration-specific rather than universal Fargate prices.

Consequently, the cost structures can be characterized as:

- **Kubernetes:** infrastructure + platform operations, with optional managed-service fees.
- **Swarm:** infrastructure + platform operations.
- **Fargate:** metered container compute + associated AWS services.

Industry adoption and ecosystem

Kubernetes has a substantially documented cloud-native ecosystem and broad production adoption. The supplied CNCF survey provides evidence of this adoption within its surveyed population, reporting 82% production Kubernetes use among container users.

Docker Swarm's main architectural advantage is its integration with the Docker Engine ecosystem. Its concepts remain closely tied to Docker's container and service model.

Fargate's ecosystem is primarily the **AWS container ecosystem**, particularly ECS and EKS. Its value proposition therefore depends substantially on the surrounding AWS infrastructure and services with which the workloads are integrated.

The ecosystems are consequently structurally different rather than simply different versions of the same technology:

- Kubernetes emphasizes an **extensible orchestration ecosystem**.
- Swarm emphasizes **Docker-native orchestration**.
- Fargate emphasizes **managed AWS container infrastructure**.

Final Verdict

The three technologies represent **three different approaches to container operations**, rather than three interchangeable implementations of exactly the same architecture.

Kubernetes provides the broadest orchestration model in the supplied fact set. It exposes substantial control over cluster architecture and workload scheduling, supports large documented cluster targets, and has extensive industry adoption. That breadth also means that self-managed deployments introduce significant infrastructure and operational responsibilities.

Docker Swarm integrates orchestration directly into Docker Engine. Its manager/worker architecture, service model, and built-in networking and service-management capabilities provide a comparatively compact operational model. Its documented scalability guidance is less extensive than Kubernetes' published large-cluster specifications, and manager scaling is constrained by the requirements of Raft consensus.

AWS Fargate fundamentally changes the infrastructure-management model. Rather than asking the customer to operate container hosts, it supplies managed compute for ECS and EKS workloads. This substantially reduces host-level administration, while shifting the operational focus toward AWS services, resource configuration, networking, IAM, and consumption-based billing.

From an architectural perspective, the key distinction is therefore **control versus abstraction**: Kubernetes provides extensive infrastructure and orchestration control, Swarm provides Docker-integrated orchestration with a simpler architectural footprint, and Fargate abstracts much of the underlying compute infrastructure behind a managed service model. The appropriate choice depends on the organization's required control boundary, workload characteristics, operational model, existing infrastructure, and cost constraints rather than on a single universal technical criterion.